95
Pass

Pavan Kumar Yalavarthi

PDF generated at: 25 Dec 2025 15:27:52 UTC

View this report on HackerRank

Score

95% • 190.0 / 200
scored in eClerx - Java Coding GCC Test - 4+ Years in 63 min 37 sec on 25 Dec 2025 19:50:55 IST

Candidate Information

Email	pavan.k.yalavarthi@icloud.com
Test	eClerx - Java Coding GCC Test - 4+ Years
Candidate Packet	View
Taken on	25 Dec 2025 19:50:55 IST
Time taken	63 min 37 sec/ 136 min
Work Experience	4 years
Invited by	Kalyani
Invited on	23 Dec 2025 18:17:24 IST

Skill Distribution

No.	Skill	Score
1	Java Intermediate	95%
2	SQL Intermediate	100%
3	Java Basic	100%

4	Selenium Intermediate	100%
5	Application Security	100%
6	Spring Boot Intermediate	100%

Tags Distribution

Java	100%	Collections	100%
Queue	100%	Medium	97%
Stacks	100%	java networking	0%
Queues	100%	Arrays	100%
Vectors	100%	Language Proficiency	100%
Problem Solving	100%	Overloading	100%
OOPS	100%	Polymorphism	100%
Programming	100%	MySQL	100%
Easy	66%	Selenium	100%
XPath	100%	Diode	0%
Semiconductor	0%	Payloads	100%
Metasploit	100%	Inheritance	100%
Exception Handling	100%	Spring Boot	100%
Spring Security	100%	Actuator	100%
Application Metrics	100%		

Questions

Java (Intermediate) • 20 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Empty Collection Multiple Choice	Java (Intermediate)	5/5	-	1 min 50 sec	✓
2	Java Networking Multiple Choice	Java (Intermediate)	0/5	-	1 min 25 sec	✗
3	Java PriorityQueue Multiple Choice	Java (Intermediate)	5/5	-	1 min 32 sec	✓
4	Java : Collections Multiple Choice	Java (Intermediate)	5/5	-	48 sec	✓
5	Java : OOPS Multiple Choice	Java (Intermediate)	5/5	-	17 sec	✓

SQL (Intermediate) • 20 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	MySQL: Group By Order Multiple Choice	SQL (Intermediate)	5/5	-	57 sec	✓
7	Java Types Multiple Choice	Java (Basic)	5/5	-	7 sec	✓
8	Selenium XPath Multiple Choice	Selenium (Intermediate)	5/5	-	44 sec	✓
9	Diode Basics and Equivalent Circuits Multiple Choice	-	0/5	-	16 sec	✗
10	Concealed Payloads in Windows Environments Multiple Choice	Application Security	5/5	-	33 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
11	Java: Task Execution with Semaphore Coding	Java (Intermediate)	75/75	-	17 min 9 sec	✓

Spring Boot (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
12	Spring Boot: Metrics Management & Security Validation Back-end Developer	Spring Boot (Intermediate)	75/75	-	37 min 42 sec	✓

1. Empty Collection

Correct

Multiple Choice | Java | Collections | Queue | Medium | Stacks

Question description

Which code displays an empty (`[]`) collection when executed?

Interviewer guidelines

The correct answer is D.

There are 10 items in the stack initially. In the first loop, we add these 10 items to the stack. In the second loop, we iterate based on the size of the stack, and in every iteration, we remove the item from the top of the stack and insert it into the queue. Then decrement the counter. This is a very important step as this will adjust the index counter and will remove all the items from the stack.
For example, if i is not decremented, then the stack size is reduced and i is incremented. After 4th iteration, the value of the variable I will be greater than the size of the stack and exit from the loop.
But the stack will not be empty. The final elements in the stack are mentioned below:

[5, 6, 7, 8, 9]

To avoid this, we need to decrement the value of the variable i so that it will never increment. It will be always '0'. But the loop will exit when the stack becomes empty as the condition is evaluated to false.

The same case is applicable to the third loop as well as the variable i is required to be reduced. This way it will remove all the items from the queue.

Based on the above explanation, other answers are not correct.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] <pre> <code class="language-java">Stack<Integer> stack = new Stack<>(); Queue queue = new LinkedList(); for (int i = 0; i < 10; i++) stack.add(i); for (int i = 0; i < stack.size() - 1; i++) { int n = stack.remove(0); queue.add(n); } for (int i = 0; i < queue.size() - 1; i++) { int n = (int) queue.remove(); i = i - 1; } System.out.println(queue);</code></pre>

- [ ] <pre> <code class="language-java">Stack<Integer> stack = new Stack<>(); Queue queue = new LinkedList(); for (int i = 0; i < 10; i++) stack.add(i); for (int i = 0; i < stack.size(); i++) { int n = stack.remove(0); queue.add(n); } for (int i = 0; i < queue.size(); i++) { int n = (int) queue.remove(); } System.out.println(queue); </code></pre>

- [ ] <pre> <code class="language-java">Stack<Integer> stack = new Stack<>(); Queue queue = new LinkedList(); for (int i = 0; i < 10; i++) stack.add(i); for (int i = 0; i < stack.size(); i++) { int n = stack.remove(0); queue.add(n); i = i - 1; } for (int i = 0; i < queue.size(); i++) { int n = (int) queue.remove(); } System.out.println(queue); </code></pre>

- [x] **<pre> <code class="language-java">Stack<Integer> stack = new Stack<>(); Queue queue = new LinkedList(); for (int i = 0; i < 10; i++) stack.add(i); for (int i = 0; i < stack.size(); i++) { int n = stack.remove(0); queue.add(n); i = i - 1; } for (int i = 0; i < queue.size(); i++) { int n = (int) queue.remove(); i = i - 1; } System.out.println(queue); </code></pre>   **

No comments.

2. Java Networking

Incorrect

Multiple Choice | java networking | Medium

Question description

This code sends a message from the client to the server.

```
import java.io.*;
import java.net.*;
public class Server {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(9999);
            Socket socket = serverSocket.accept();
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            String message = in.readLine();
            System.out.println("Client: " + message);
            socket.close();
            serverSocket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

What is the output on the server side?

Interviewer guidelines

Since we have closed the connection on the client side before sending the data. According to the Java API documentation for the BufferedReader class (typically used to read data from a socket's input stream), the readLine() method returns null if the end of the stream has been reached, which can occur if there is no input data to read.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ● Client: Hello
- ○ SocketException: Connection Closed

NullPointerException

Client: null

No comments.

3. Java PriorityQueue

Correct

Multiple Choice | Collections | Queues | Medium

Question description

What is the output of the following code?

```
import java.util.*;
public class Main{
   public static void main(String args[]){
      Queue<String> pq = new PriorityQueue<>();
      pq.add("Hello");
      pq.add("Hackerrank");
      String s1=pq.poll();
      pq.remove();
      System.out.println(s1);
      String s2=pq.remove();
      System.out.println(s2);
   }
}
```

Interviewer guidelines

First, we added two elements in the queue, “Hello” and “Hackerrank”. We removed the front element “Hackerrank” of the queue using the poll() method and saved in s1 and print it. We removed the remaining element ”Hello”. Now the queue is empty. When we call the remove() method on an empty queue it throws a “NoSuchElementException” exception. However, the poll() method does not.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] Hackerrank NoSuchElementException
- [ ] Hackerrank Hello
- [ ] Hackerrank null
- [ ] Hello Hackerrank

No comments.

---

4. Java : Collections

Correct

Multiple Choice | Medium | Java | Collections | Arrays | Vectors | Language Proficiency | Problem Solving

Question description

Which of the statements is true about ArrayList and Vector in Java?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] Vector can be resized while ArrayList cannot be
- [x] Vector is synchronized while ArrayList is not
- [ ] ArrayLists can grow but cannot shrink in size, while Vector can both grow and shrink

- Vectors allow duplicate values while ArrayList doesnot

No comments.

5. Java : OOPS

Correct

Multiple Choice | Java | Overloading | OOPS | Polymorphism | Language Proficiency | Problem Solving | Programming | Medium

Question description

What feature allows different methods to have the same name and arguments type, but a different implementation is called?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- overloading
- **overriding** ✓
- Java does not permit methods with same name and type signature
- None of the above

No comments.

6. MySQL: Group By Order

Correct

Multiple Choice | MySQL | Medium

Question description

Which expression causes a MySQL error?
Select all that apply.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ( ) SELECT is_active, COUNT(*) FROM transactions GROUP BY is_active ORDER BY 2
- ( ) SELECT is_active, COUNT(*) FROM transactions GROUP BY is_active ORDER BY COUNT(*)
- ( ) SELECT is_active, COUNT(*) AS transactions FROM transactions GROUP BY is_active ORDER BY transactions
- (●) none of the above, all expressions are correct ✓

No comments.

---

7. Java Types

Correct

Multiple Choice | Easy | Java

Question description

Which of the following is not a Java primitive type?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] String
- [ ] float
- [ ] double
- [ ] int
- [ ] byte

No comments.

8. Selenium XPath

Correct

Multiple Choice | Selenium | Medium | XPath

Question description

Which of the following XPath expressions finds all ReCAPTCHA input elements on the page?

Note: The ReCAPTCHA input elements have g-recaptcha-response in the ID attribute.

Interviewer guidelines

Relative xpath starts with // and requires tag name for correctly finding input element.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] //*[@id='g-recaptcha-response']

- [x] //input[@id='g-recaptcha-response']
- [ ] /html/body/input[@id='g-recaptcha-response']
- [ ] /html/body/*[@id='g-recaptcha-response']

No comments.

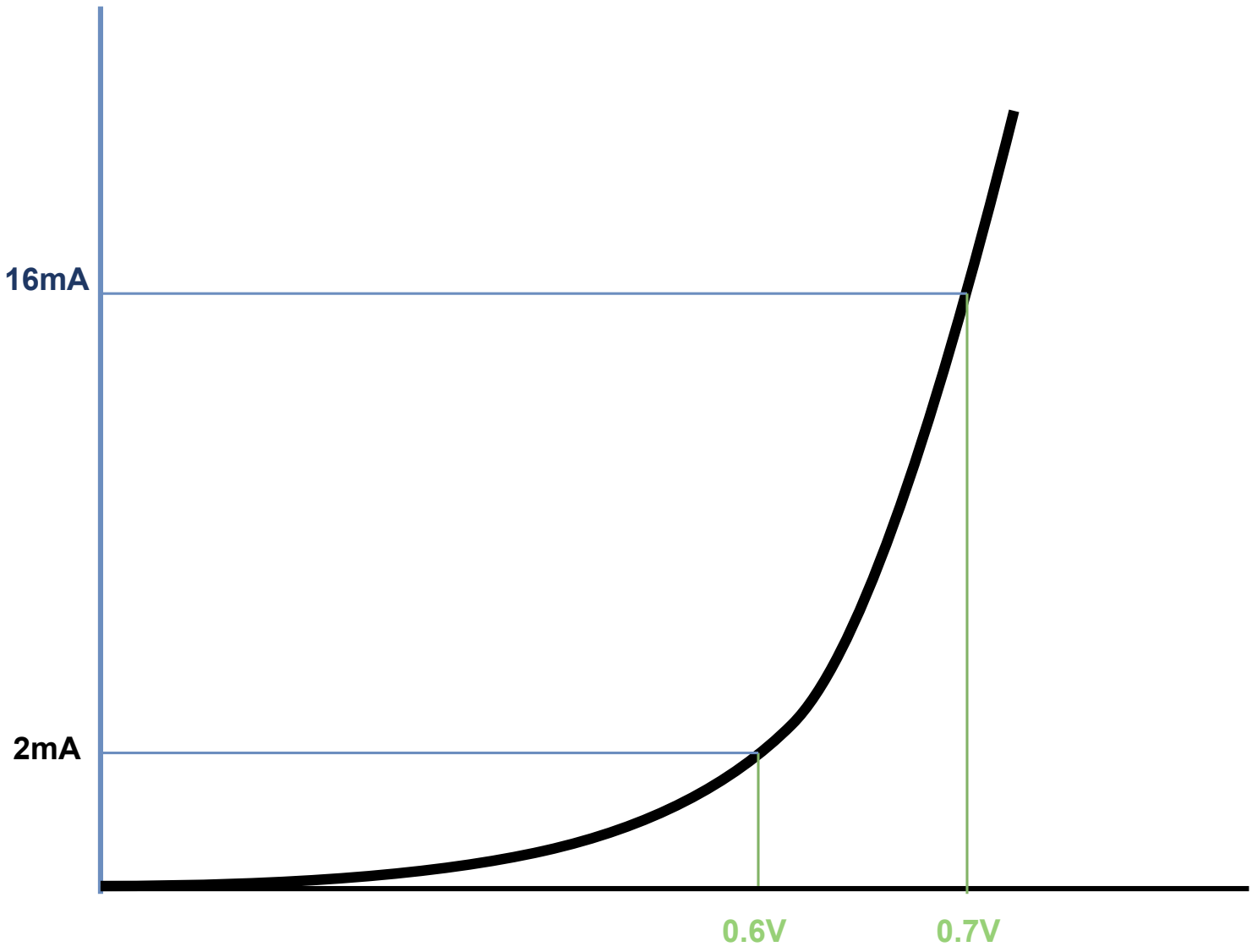
9. Diode Basics and Equivalent Circuits

Incorrect

Multiple Choice | Easy | Diode | Semiconductor

Question description

An engineer is developing a sensor array for an embedded system that includes a new silicon diode among its components. The system relies on precise readings from this array to make real-time decisions through its software. To ensure the diode's performance aligns with the system's requirements, the engineer is provided with the I-V characteristics curve of the diode. The task is to approximate the average AC resistance, r_{av}, over a specified segment of this curve. This resistance calculation is critical as it will directly influence the logic in the system's software, affecting how the embedded system interprets sensor data and makes operational decisions. Using just the curve for reference, determine the *rav* between the marked points. What is the estimate?



Interviewer guidelines

r_{av} is the average AC resistance found between two marked points in the I-V graph. Here r_{av}=0.1/14mA=7Ω.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ Not enough details
- ● 11.2 Ohms

- 6 Ohms
- 7 Ohms ✓

No comments.

10. Concealed Payloads in Windows Environments

Correct

Multiple Choice | Easy | Payloads | Metasploit

Question description

In a scenario where the target Windows system is heavily monitored, a payload must execute covertly.

Which Metasploit payload allows for stealthy execution by injecting itself into a legitimate process to maintain low visibility?

Interviewer guidelines

For discreet execution on a monitored Windows system, the "windows/meterpreter/inject_dll" payload is designed to inject the Meterpreter DLL into a legitimate process, ensuring covert operation.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ● windows/meterpreter/inject_dll ✓
- ○ windows/shell_hide_payload
- ○ meterpreter/windows/covert_execution
- ○ payload/windows/stealthy_inject

No comments.

11. Java: Task Execution with Semaphore

Correct

Coding | Java | Problem Solving | Medium | Inheritance | Exception Handling

Question description

Design a class to manage the execution of calculation tasks using semaphores. The class ensures that resources (permits) are properly reserved, tasks are executed efficiently, and resources are released after use. The class should handle two types of tasks: division and multiplication.

The implementation will allow users to:

- Reserve a specific number of calculation resources (permits) needed for task execution.
- Execute a task
- Release calculation resources

Implement the logic for the *CalculationSemaphore* class, a subclass of the *Semaphore* class, as detailed below. This class manages resource acquisition and execution of calculation tasks, ensuring that enough *permits* are available to perform division and multiplication tasks. Division tasks require two *permits* and multiplication tasks require one. After division tasks, its permits are released back to the semaphore. Multiplication consumes its permit.

The application will receive a list of strings *commands[n]*, where:

- The first string represents the number of *permits* used to initialize the *CalculationSemaphore* class.
- Each of the subsequent strings represents a calculation task to be executed. These tasks consist of an action (either "divide" or "multiply") followed by two operands, *a* and *b*, separated by spaces.

The following classes and methods are already provided:

- The abstract class *CalculationTask* represents a generic calculation task with two operands, *a* and *b*.
  - *CalculationTask(int a, int b):* initializes a *CalculationTask* object with two integers, *a* and *b*, which are the operands for the calculation task. These values are stored as instance variables.
  - *calculate() -> int:* performs specific arithmetic operations on the instance variables *a* and *b* and returns the integer result of these operations.
- *MultiplicationTask* is a subclass of *CalculationTask*.
  - *MultiplicationTask(int a, int b):* initializes a *MultiplicationTask* object with two integers, *a* and *b*, which are the operands for the calculation task $a \times b$. These values are stored as instance variables.
  - *calculate() -> int:* performs multiplication of the instance variables *a* and *b* and returns the integer result of this operation.

- *DivisionTask* is a subclass of *CalculationTask*.
  - *DivisionTask(int a, int b):* initializes a *DivisionTask* object with two integers, *a* and *b*, which are the operands for the calculation task. These values are stored as instance variables.
  - *calculate() -> int:* performs division of the instance variables *a* and *b* as $a \div b$, and returns integer result of this operation.
- The *TasksExecutor* class executes calculation tasks using a *CalculationSemaphore* to manage resource acquisition and task execution.
  - *executeTask(String action, int a, int b) -> int:* Creates a *CalculationTask* based on the given *action* and operands *a* and *b*, attempts to acquire the necessary *permits*, and if successful, executes the task and returns the result; otherwise, it returns *null* if *permits* could not be acquired or 0 if the action is invalid.

Example

Input:

```
4
multiply 1 2
multiply 1 3
divide 2 1
```

Output:

```
2
3
2
```

Explanation:

- The program initializes *CalculationSemaphore* with 4 *permits*.
- It executes a multiplication task with operands 1 and 2, resulting in a product of 2. This operation used 1 permit, and there are 3 *permits* left.
- Another multiplication task with operands 1 and 3 is executed, yielding a product of 3. There are 2 *permits* left.
- A division task with operands 2 and 1 is executed, resulting in a quotient of 2. The *permits* are released back to the semaphore. There are 2 *permits* left.

Class Description

Complete the class *CalculationSemaphore* in the editor below.

Methods

- *CalculationSemaphore*(*int permits*): Initializes the semaphore with the specified number of *permits*, controlling access to resources for calculation tasks. This is provided in the code stub.
- *acquireResourcesForTask(CalculationTask task) -> boolean*: Acquires the necessary *permits* from the semaphore based on whether the *task* is a division (requires two *permits*) or multiplication (requires one *permit*), returning *true* if there are enough *permits* and *false* if there are not.
- *executeTask(CalculationTask task) -> int*: Performs the calculation of the given *task* and then releases two *permits* to the semaphore if it is a division task; handles errors gracefully by returning zero if there is an exception during

calculation.

Note: The implementation should release *permits* only for division tasks.

Constraints

- $0 \le a \le 10$
- $0 \le b \le 10$
- $0 \le$ number of *permits* ≤ 10
- $1 \le$ the size of the array *commands* ≤ 10

INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, the size of the array *commands*.
The second line contains the first element of *commands.* It is an integer *p* as a String, denoting the number of *permits* available for the *CalculationSemaphore*.
Each of the following lines contains 3 space-separated words, denoting *commands[i]*.

SAMPLE CASE 0

Sample Input 0

```
5
2
divide 2 0
multiply 1 1
multiply 1 2
multiply 1 3
```

Sample Output 0

```
0
1
2
```

Explanation

The program executed the following:

- The program initializes *CalculationSemaphore* with 2 *permits*.
- The program attempts to execute a division task with operands 2 and 0, resulting in a division by zero exception. The exception is caught and *permits* released. The method returns 0. Now *permits = 2*.
- The program executes the multiplication task with operands 1 and 1, resulting in a product of 1. Now *permits = 1*.
- The program executes the multiplication task with operands 1 and 2, resulting in a product of 2. Now *permits = 0*.
- Because the program does not have enough *permits*, the next task is not executed.

SAMPLE CASE 1

Sample Input 1

```
4
2
divide 2 1
multiply 1 2
multiply 1 3
```

Sample Output 1

```
2
2
3
```

Explanation

The program executed the following:

- The program initializes *CalculationSemaphore* with 2 *permits*.
- The program executes a division task with operands 2 and 1, resulting in a quotient of 2. *Permits* are released.
- The program executes a multiplication task with operands 1 and 2, resulting in a product of 2. Now *permits* = 1.
- The program executes a multiplication task with operands 1 and 3, resulting in a product of 3. Now *permits* = 0.

Interviewer guidelines

▼ SOLUTION

Skills: Inheritance, Semaphore, Exception Handling

Complete Solution:

```
class CalculationSemaphore extends Semaphore {

    public CalculationSemaphore(int permits) {
        super(permits);
    }

    // Acquires the necessary permits based on the type of task
    public boolean acquireResourcesForTask(CalculationTask task) {
        int requiredPermits = (task instanceof DivisionTask) ? 2 : 1;
        return tryAcquire(requiredPermits); // Attempt to acquire permits
    }

    // Executes the task and handles exceptions if they occur
    public int executeTask(CalculationTask task) {
        try {
            int result = task.calculate(); // Perform calculation

            if (task instanceof DivisionTask) {
                release(2); // Release permits after division task
            }
            return result;
```

```
        } catch (ArithmeticException e) {
            if (task instanceof DivisionTask) {
                release(2); // Release permits on division error
            }
            return 0; // Handle division by zero gracefully
        }
    }
}
```

▼ COMPLEXITY ANALYSIS

Time Complexity - big O

Space Complexity - big O

Candidate's Solution

Language used: Java 21

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

abstract class CalculationTask {
    protected int a;
    protected int b;

    public CalculationTask(int a, int b) {
        this.a = a;
        this.b = b;
    }

    public abstract int calculate();
}



class MultiplicationTask extends CalculationTask {

    public MultiplicationTask(int a, int b) {
        super(a, b);
    }

```

```
    @Override
    public int calculate() {
        return a * b;
    }
}

class DivisionTask extends CalculationTask {

    public DivisionTask(int a, int b) {
        super(a, b);
    }

    @Override
    public int calculate() {
        return (a / b);
    }
}

class CalculationSemaphore extends Semaphore {

    public CalculationSemaphore(int permits) {
        super(permits);
    }

    private int requiredSemaphores(CalculationTask task) {
        if(task instanceof MultiplicationTask)
            return 1;
        else if(task instanceof DivisionTask)
            return  2;
        return 0;
    }

    //todo : implement
    public boolean acquireResourcesForTask(CalculationTask task) {
        int required = requiredSemaphores(task);
        if (required <= this.availablePermits()) {
            try {
                this.acquire(required);
                return true;
            } catch (Exception e) {

            }
        }
        return false;
    }

    //todo : implement
    public int executeTask(CalculationTask task) {

        int result = 0;
        try {
            result = task.calculate();
        } catch (Exception e) {
```

```java

        }
        if (task instanceof DivisionTask) {
            try {
                this.release(2);
            } catch (Exception e) {

            }
        }
        return result;
    }
}

class TasksExecutor {
    private CalculationSemaphore calculationSemaphore;

    TasksExecutor(CalculationSemaphore calculationSemaphore) {
        this.calculationSemaphore = calculationSemaphore;
    }

    public Integer executeTask(String action, int a, int b) {
        CalculationTask task = null;
        if (action.equals("divide")) {
            task = new DivisionTask(a, b);
        } else if (action.equals("multiply")) {
            task = new MultiplicationTask(a, b);
        }

        if (task != null) {
            boolean wasAcquired = calculationSemaphore.acquireResourcesForTask(task);
            if (!wasAcquired) {
                return null;
            }
            return calculationSemaphore.executeTask(task);
        } else {
            return 0;
        }
    }
}

public class Solution {

    public static List<Integer> executeCommands(List<String> commands) {
        String permitsCount = commands.get(0);
        CalculationSemaphore calculationSemaphore = new CalculationSemaphore(Integer.valueOf(permitsCount));
        TasksExecutor tasksExecutor = new TasksExecutor(calculationSemaphore);

        List<Integer> commandsResults = new ArrayList<>();

        for (int i = 1; i < commands.size(); i++) {
            String command = commands.get(i);
            String[] commandParts = command.split(" ");
```

```
            String action = commandParts[0].trim();
            int a = Integer.valueOf(commandParts[1].trim());
            int b = Integer.valueOf(commandParts[2].trim());
            Integer result = tasksExecutor.executeTask(action, a, b);
            if (result != null) {
                commandsResults.add(result);
            }
        }
        return commandsResults;
    }

    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int commandsCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<String> commands = new ArrayList<>();

        for (int i = 0; i < commandsCount; i++) {
            String commandsItem = bufferedReader.readLine();
            commands.add(commandsItem);
        }

        List<Integer> result = executeCommands(commands);

        for (int i = 0; i < result.size(); i++) {
            bufferedWriter.write(String.valueOf(result.get(i)));

            if (i != result.size() - 1) {
                bufferedWriter.write("\n");
            }
        }

        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample	Success	1	0.041 sec	36.7 KB
Testcase 1	Easy	Sample	Success	1	0.0376 sec	36.7 KB

Testcase 2	Easy	Sample	Success	1	0.0368 sec	36.8 KB
Testcase 3	Easy	Hidden	Success	2	0.0412 sec	36.6 KB
Testcase 4	Easy	Hidden	Success	5	0.04 sec	36.7 KB
Testcase 5	Easy	Hidden	Success	5	0.0363 sec	36.5 KB
Testcase 6	Medium	Hidden	Success	10	0.0388 sec	36.5 KB
Testcase 7	Medium	Hidden	Success	10	0.0453 sec	36.8 KB
Testcase 8	Medium	Hidden	Success	15	0.0348 sec	36.8 KB
Testcase 9	Medium	Hidden	Success	10	0.0473 sec	36.8 KB
Testcase 10	Medium	Hidden	Success	10	0.0439 sec	36.6 KB
Testcase 11	Easy	Hidden	Success	5	0.0364 sec	36.2 KB

No comments.

12. Spring Boot: Metrics Management & Security Validation

Correct

Back-end Developer | Spring Boot | Medium | Spring Security | Actuator | Application Metrics

Question description

There is a tiny student information system built on Spring Boot. The admin wants to know some application metrics and metadata of the system running the application. However, they don't want to reveal to anyone that the application is monitoring information, which means this information should be protected.

The definitions and detailed requirements list follow. You will be graded on whether your application performs data retrieval and manipulation based on given use cases exactly as described in the requirements.

Here are the application requirements:

1. Write a user-defined actuator function that provides the Spring version information of the system. This information should be served via URL, like `/monitoring/springVersion`.
2. Write a custom Spring Boot security function to protect the URL `/monitoring/springVersion`.
3. All the default and system-provided actuator endpoints must be disabled.
4. Allow only the custom actuator endpoint `/monitoring/springVersion`.
5. The rest of the app URLs should work without authentication.

While securing the actuator endpoint, note the following:

- Session management needs to be *stateless*.
- CSRF needs to be disabled.

The application should expose the following endpoints:

`GET` request to `/monitoring/springVersion`:

- this is the actuator endpoint
- it must require basic HTTP authentication to be accessible
- it should return the Spring version used in this app as a response string with status code 200

`GET` request to `/app/student`:

- this is the REST endpoint
- it should be accessible without authentication
- implementation for this has already been provided

The authentication features in this question are handled using the HTTP Basic Authentication. For any given request to be authenticated, the header `"Authorization: Basic {encoded user and password}"` is passed. Generally, in a full-fledged web application, this password would be encoded, but for the scope of this question, the password will be a plain-text string.

Your task is to complete the implementation of the `SpringVersionActuator` & `SpringSecurityConfig` classes so it passes all the test cases when running the provided unit tests.

Note that the username and password combinations have been configured as in-memory values in `SpringSecurityConfig.java`. You just have to handle the part of defining the version actuator endpoint and securing it.

▼ EXAMPLE REQUESTS AND RESPONSES

`GET` request to `/monitoring/springVersion`

Header: Authorization = "Basic admin:admin"

The response code is 200, and the response is:

```
{
  "version": "2.2.1.RELEASE"
}
```

`GET` request to `/monitoring/springVersion`
Header: Authorization = "Basic wrongusername:admin"
The response code is 401, "Unauthorized".

`GET` request to `/actuator/info`
Header: Authorization = "Basic: admin:admin"
The response code is 404, "Not Found".

`GET` request to `/app/student`
The response code is 200, and the response body is `Student`.

Candidate's Submission

Testcase	Test file	Status	Score
shouldGet404SinceNonExposedEndpoints	TEST-com.hackerrank.monitoring.MonitoringTests.xml	Success	15.0 / 15.0
shouldGetResult	TEST-com.hackerrank.monitoring.MonitoringTests.xml	Success	15.0 / 15.0
shouldGetSinceNonActuatorEndpoint	TEST-com.hackerrank.monitoring.MonitoringTests.xml	Success	15.0 / 15.0
shouldGetAuthorized	TEST-com.hackerrank.monitoring.MonitoringTests.xml	Success	15.0 / 15.0
validateBasePathAndActuatorEndpoint	TEST-com.hackerrank.monitoring.MonitoringTests.xml	Success	15.0 / 15.0

Review logs: output log
View candidate code

No comments.